Python and Git 101

Python Programming and Data Analysis · Day 1 · Dataminded Academy

dataminded

Welcome to Day 1

Your trainers

- **Jan Vanbuel**, Dataminded
- **Oliver Willekens**, Dataminded, assisting in the room

Today

- Development environment
- Programming fundamentals
- Version control with Git

Ground rules

- Ask early, ask often
- Stuck for 5 minutes? Wave at us
- No programming experience assumed



What you can do by 17:00

- Set up a Python environment and run a script from PowerShell
- Create a virtual environment per project and install packages with uv
- Write a script with functions, conditionals and loops, import a package into it, and run it from VS Code
- Read a stack trace and find the error it points to
- Automate a repetitive task with a script that reads and writes files
- Clone an Azure DevOps repository, commit on a branch, push and pull



Agenda

01	09:00 Setup hour
02	10:00 Scripts, notebooks and running Python
03	10:45 Language fundamentals, lunch at 12:30
04	13:15 Standard library, files and exceptions
05	14:15 Projects with uv and the automation capstone
06	15:15 Version control and Git
07	16:45 Wrap-up



Setup hour: check your laptop

```
uv --version
uv python list --only-installed
code --version
git --version
```

Something fails?

Do not debug alone. Call a trainer, setup problems are normal on managed laptops.

Scripts and notebooks



A script is a text file that runs top to bottom

```
# bench_summary.py
pressure_bar = 7.5

power_kw = 37.5
print(f"Power: {power_kw} kW at {pressure_bar} bar")

print("Done")
```

```
Power: 37.5 kW at 7.5 bar
Done
```




A notebook mixes code, results and text

Cells

- Code cells and text cells
- Run one cell at a time
- Output appears under the cell: numbers, tables, plots

Kernel

- A running Python process behind the notebook
- Keeps all variables in memory between cells
- Restart it and the memory is gone



Scripts versus notebooks

	Script (.py)	Notebook (.ipynb)
Runs	Start to finish, every time, on demand or scheduled	Cell by cell, in any order you click
Great for	Automation, reusable tools, anything others run	Exploring data, plotting, explaining results
Weak at	Slow feedback when exploring	Repeatability and testing
In Git	Clean text, easy to compare versions	JSON with embedded output, noisy differences



The notebook trap: hidden state

```
# Cell 1                    # Cell 2                    # Cell 3
limit_bar = 9.0             limit_bar = 6.0             print(limit_bar)
```

- You ran cell 1, then cell 2, then edited cell 2 away and forgot about it
- Cell 3 prints a value that no longer appears anywhere in the visible code
- A colleague runs the notebook top to bottom and gets a different answer

Explore in a notebook, deliver in a script



Rule of thumb: once you would run it twice, it belongs in a script.

Python on your laptop



The interpreter runs your file



Python is a program. Your `.py` file is just text it reads and executes.

Installing Python on a managed laptop with uv

```
uv python install 3.13
uv python list --only-installed
```

- No administrator rights needed: uv installs Python in your user profile
- Several Python versions can live side by side
- Each project chooses its version, uv picks the right one

What is the PATH?

```
$env:Path -split ';'
```

```
C:\Users\you\.local\bin
C:\Windows\system32
C:\Windows
C:\Program Files\Git\cmd
...
```

Type a command, and Windows searches these folders, top to bottom, for a matching program.



Which Python am I running?

```
Get-Command python
```

```
CommandType  Name        Source
-----------  ----        ------
Application  python.exe  C:\Users\you\AppData\Local\Microsoft\WindowsApps\python.exe
```

Watch out

A `WindowsApps` path is the Microsoft Store shortcut, not a real Python. It opens the Store or does nothing useful.

Run a script from PowerShell

```
# hello.py
print("Compressor test bench online")
```

```
cd C:\Users\you\training
uv run python hello.py
```

```
Compressor test bench online
```

Try ideas in the REPL

```
uv run python
```

```
>>> 7.5 * 14.5038
108.7785
>>> pressures = [6.8, 7.1, 7.5]
>>> max(pressures)
7.5
>>> exit()
```




Running from VS Code

1

OPEN THE FOLDER

File, Open Folder, choose your project folder

2

SELECT THE INTERPRETER

Command Palette, Python: Select Interpreter, pick the one in `.venv` or the uv Python 3.13

3

RUN

Play button at the top right, or `uv run python hello.py` in the integrated terminal



Exercise 01: First script

`exercises/python/01-first-script`



Language fundamentals



Variables are names for values

```
pressure_bar = 7.5
temperature_c = 82.3
run_id = "TB-014"

pressure_bar = 8.0
print(pressure_bar)
print(temperature_c)
```

```
8.0
82.3
```




The basic data types

```
run_id = "TB-014"        # str: text
cycles = 12              # int: whole number
flow_l_s = 18.4          # float: decimal number
is_stable = True         # bool: True or False
operator = None          # None: no value (yet)

print(type(flow_l_s))
```

```
<class 'float'>
```

Arithmetic, and a float surprise

```
print(7 / 2)       # 3.5   true division
print(7 // 2)      # 3     whole part only
print(7 % 2)       # 1     remainder
print(2 ** 10)     # 1024  power, not ^
print(0.1 + 0.2)   # 0.30000000000000004
```

Floats are binary approximations. Never test two floats for exact equality.

Strings and f-strings

```
run_id = "TB-014"
pressure_bar = 7.5
print(f"Run {run_id}: {pressure_bar:.1f} bar")
print(f"{pressure_bar * 14.5038:.0f} psi")
print("TB" in run_id, run_id.lower())
```

```
Run TB-014: 7.5 bar
109 psi
True tb-014
```




Converting between types

```
text = "7.5"
pressure_bar = float(text)
print(pressure_bar + 1)        # 8.5

count = 3
print("Cycles: " + count)
```

```
TypeError: can only concatenate str (not "int") to str
```




Name things so the unit is visible

Hard to read

```
p = 7.5
t = 82.3
q = 18.4
x1 = p * q
```

Clear

```
pressure_bar = 7.5
temperature_c = 82.3
flow_l_s = 18.4
power_kw = 37.5
```

Lowercase with underscores (`snake_case`), and the unit as a suffix.



Exercise 02: Variables and types

`exercises/python/02-variables-and-types`



Lists: an ordered collection

```
pressures_bar = [6.8, 7.1, 7.5, 7.4, 7.9]
print(pressures_bar[0])
print(len(pressures_bar))
pressures_bar.append(8.2)
print(max(pressures_bar))
```

```
6.8
5
8.2
```




Indexing starts at zero

index	0	1	2	3	4
value	6.8	7.1	7.5	7.4	7.9
negative	-5	-4	-3	-2	-1

```
p = [6.8, 7.1, 7.5, 7.4, 7.9]
print(p[1:3], p[:2], p[-2:])
```

```
[7.1, 7.5] [6.8, 7.1] [7.4, 7.9]
```

Everyday list operations

```
p = [7.5, 6.8, 7.9, 7.1]
print(sum(p) / len(p))
print(sorted(p))
print(7.9 in p)
p.remove(6.8)
print(p)
```

```
7.325
[6.8, 7.1, 7.5, 7.9]
True
[7.5, 7.9, 7.1]
```




Tuples and sets

Tuple: fixed

```
limits = (5.0, 9.0)
low, high = limits
print(low, high)
limits[0] = 4.0
```

```
TypeError: 'tuple' object does not
support item assignment
```

Set: unique

```
runs = {"TB-014", "TB-015", "TB-014"}
print(len(runs))
print("TB-015" in runs)
```

```
2
True
```

Dictionaries: key to value

```
reading = {"run": "TB-014", "pressure_bar": 7.5}
print(reading["pressure_bar"])
reading["flow_l_s"] = 18.4
print(reading.get("power_kw"))
print(reading)
```

```
7.5
None
{'run': 'TB-014', 'pressure_bar': 7.5, 'flow_l_s': 18.4}
```




A table of readings: a list of dicts

```
readings = [
    {"t_s": 0, "pressure_bar": 6.8},
    {"t_s": 10, "pressure_bar": 7.1},
    {"t_s": 20, "pressure_bar": 7.5},
]
print(readings[2]["pressure_bar"])
print(len(readings))
```

```
7.5
3
```

One dict per row. This is exactly the shape you get when you read a CSV file.



Exercise 03: Collections

`exercises/python/03-collections`



Decisions with if, elif, else

```
pressure_bar = 9.4
if pressure_bar > 9.0:
    print("ALARM: over limit")
elif pressure_bar < 5.0:
    print("Warning: low pressure")
else:
    print("OK")
```

```
ALARM: over limit
```




Comparisons and logic

```
pressure_bar = 7.5
run_id = "TB-014"
print(pressure_bar == 7.5, pressure_bar != 7.5)
print(5.0 <= pressure_bar <= 9.0)
print(pressure_bar > 9.0 or run_id == "TB-014")
print(not run_id.startswith("TB"))
```

```
True False
True
True
False
```




Indentation is the syntax

Works

```
if pressure_bar > 9.0:
    print("ALARM")
    print("Stop the test")
print("Checked")
```

Fails

```
if pressure_bar > 9.0:
print("ALARM")
```

```
IndentationError: expected an
indented block after 'if' statement
on line 1
```




Repeating with for

```
pressures_bar = [6.8, 7.1, 9.4]
for p in pressures_bar:
    print(f"{p:.1f} bar")

for i in range(3):
    print(i)
```

```
6.8 bar
7.1 bar
9.4 bar
0
1
2
```




Looping until a condition: while

```
pressure_bar = 1.0
steps = 0
while pressure_bar < 7.0:
    pressure_bar *= 1.5
    steps += 1
print(steps, round(pressure_bar, 2))
```

```
5 7.59
```

If the condition never becomes false, the loop never ends. Stop it with `Ctrl+C`.



Loop pattern: filter and summarise

```
pressures_bar = [6.8, 7.1, 9.4, 7.4, 9.1]
high = []
total = 0.0

for p in pressures_bar:
    total += p
    if p > 9.0:
        high.append(p)

print(high, round(total / len(pressures_bar), 2))
```

```
[9.4, 9.1] 7.96
```




Exercise 04: Control flow

`exercises/python/04-control-flow`



Functions: name a piece of logic

```
def to_psi(pressure_bar):
    return round(pressure_bar * 14.5038, 1)

print(to_psi(7.5))
print(to_psi(10))
```

```
108.8
145.0
```

Default values and keyword arguments

```
def is_over_limit(pressure_bar, limit_bar=9.0):
    return pressure_bar > limit_bar

print(is_over_limit(9.4))
print(is_over_limit(9.4, limit_bar=10.0))
```

```
True
False
```




`return` is not print

```
def double_and_print(x):
    print(x * 2)

result = double_and_print(4)
print(result)
```

```
8
None
```

A function without `return` gives back `None`. Use `print` to show, `return` to pass on.

Variables inside a function stay inside

```
def mean(values):
    total = sum(values)
    return total / len(values)

print(round(mean([6.8, 7.1, 7.5]), 2))
print(total)
```

```
7.13
NameError: name 'total' is not defined
```




A small program made of functions

```
def mean(values):
    """Return the arithmetic mean of a list of numbers."""
    return sum(values) / len(values)

def summarise(values):
    return {"min": min(values), "max": max(values), "mean": mean(values)}

def main():
    pressures_bar = [6.8, 7.1, 9.4, 7.4]
    print(summarise(pressures_bar))

if __name__ == "__main__":
    main()
```




Exercise 05: Functions

`exercises/python/05-functions`



The standard library



Batteries included: import and math

```
import math

diameter_m = 0.05
area_m2 = math.pi * (diameter_m / 2) ** 2
flow_m3_s = 18.4 / 1000
print(round(flow_m3_s / area_m2, 2))

print(math.isclose(0.1 + 0.2, 0.3))
```

```
9.37
True
```




pathlib: paths as objects

```python
from pathlib import Path

data_dir = Path("data")
log_file = data_dir / "run_014.csv"

print(log_file.name, log_file.suffix, log_file.stem)
print(log_file.exists())

for f in sorted(data_dir.glob("*.csv")):
    print(f)
```

```
run_014.csv .csv run_014
True
data\run_014.csv
data\run_015.csv
```




Windows paths and the backslash trap

```
print("C:\new\table.csv")
print(r"C:\new\table.csv")
```

```
C:
ew  able.csv
C:\new\table.csv
```

In a normal string, `\n` is a newline and `\t` is a tab. Use `Path(...)`, forward slashes, or a raw string with `r"..."`.

csv: reading rows

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```

```
import csv
with open("data/run_014.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows[0])
print(float(rows[1]["pressure_bar"]) + 1)
```

```
{'t_s': '0', 'pressure_bar': '6.8', 'temperature_c': '71.2'}
8.1
```




csv: writing rows

```
import csv
rows = [{"t_s": 0, "pressure_bar": 6.8}, {"t_s": 10, "pressure_bar": 7.1}]
with open("out.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["t_s", "pressure_bar"])
    writer.writeheader()
    writer.writerows(rows)
```

```
t_s,pressure_bar
0,6.8
10,7.1
```




json: settings and nested data

```
import json
config = {"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": None}
text = json.dumps(config)
print(text)
back = json.loads(text)
print(back["channels"][0], back["operator"])
```

```
{"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": null}
pressure None
```

More built-in modules worth knowing

- `statistics`: mean, median, stdev
- `datetime`: dates, times, durations
- `shutil`: copy and move files
- `argparse`: command-line options for scripts
- `collections`: Counter, defaultdict
- `random`: random numbers for simulations

Outside the standard library: packages you install, which is what uv is for.



Exercise 06: Standard library

`exercises/python/06-standard-library`



Files and exceptions



Reading a text file

```
from pathlib import Path

with open("data/run_014.csv", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

text = Path("data/run_014.csv").read_text(encoding="utf-8")
```

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```




Writing and appending

```
lines = ["run,max_pressure_bar", "TB-014,7.9"]
with open("summary.txt", "w", encoding="utf-8") as f:
    for line in lines:
        f.write(line + "\n")

with open("summary.txt", "a", encoding="utf-8") as f:
    f.write("TB-015,8.2\n")
```

Mode `"w"` replaces an existing file without asking. `"a"` adds at the end, `"r"` only reads.

A traceback is a map to the error

```
Traceback (most recent call last):
  File "convert.py", line 2, in <module>
    pressure_bar = float(text)
ValueError: could not convert string to float: 'seven'
```

- Line 1 is only a header, it says: read on
- The middle shows the file, the line number and the code that failed
- The last line names the error type and what went wrong. Read it first.



Tracebacks through several functions

```
Traceback (most recent call last):
  File "analyse.py", line 5, in <module>
    print(mean(readings))
          ~~~~^^^^^^^^^^
  File "analyse.py", line 2, in mean
    return sum(values) / len(values)
           ~~~~~~~~~~~~^~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

Read bottom up: what failed, where it failed, then who called it.



Errors you will meet

	Typical cause
FileNotFoundError	Wrong path or wrong working folder
ValueError	Right type, unusable value: `float("seven")`
TypeError	Mixing types: `"a" + 1`
KeyError	Dictionary key that does not exist, often a typo
IndexError	List position beyond the end, often an off-by-one
NameError	Misspelled name, or used before it was created



Handling errors with try / except

```
text = "seven"
try:
    pressure_bar = float(text)
except ValueError:
    pressure_bar = None
    print(f"Skipping bad value: {text!r}")
print(pressure_bar)
```

```
Skipping bad value: 'seven'
None
```




Catch specific errors and raise your own

Do

```
try:
    text = path.read_text()
except FileNotFoundError:
    print(f"Missing: {path}")
```

Don't

```
try:
    text = path.read_text()
except:
    pass
```

```
if not values:
    raise ValueError("mean() needs at least one value")
```




Exercise 07: Files and exceptions

`exercises/python/07-files-and-exceptions`



Projects with uv



One environment per project

Shared installation

- Project A needs a package at version 1
- Project B needs version 2
- Only one can be installed, one project breaks

Virtual environment per project

- Each project has its own folder `.venv`
- It holds its own Python link and its own packages
- Projects cannot disturb each other



uv and pip: the same job, fewer steps

	pip workflow	uv workflow
Environment	`python -m venv .venv`	`uv init`, then automatic
Activate	`.venv\Scripts\Activate.ps1`	Not needed: `uv run`
Install	`pip install rich`	`uv add rich`
Declare	`requirements.txt`, by hand	`pyproject.toml`, updated by uv
Pin versions	`pip freeze`, if you remember	`uv.lock`, automatic



Start a project: uv init

```
uv init bench-tools
cd bench-tools
```

```
bench-tools/
  .python-version    Python version for this project
  README.md          describe the project
  main.py            starter script
  pyproject.toml     project settings and dependencies
  .gitignore         files Git should skip
```




pyproject.toml declares what you need

```toml
[project]
name = "bench-tools"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "rich>=15.0.0",
]
```

Human readable, and edited for you by `uv add` and `uv remove`.



Add a package with uv add

```
uv add rich
```

```
Resolved 4 packages in 2s
Installed 4 packages in 5ms
 + markdown-it-py==4.3.0
 + mdurl==0.1.2
 + pygments==2.21.0
 + rich==15.0.0
```

One command: creates `.venv` if needed, installs, updates `pyproject.toml` and `uv.lock`.



uv.lock pins exact versions

```
[[package]]
name = "rich"
version = "15.0.0"
source = { registry = "https://pypi.org/simple" }
...
```

pyproject.toml

Says what you need: `rich>=15.0.0`

uv.lock

Records what you got: `rich 15.0.0`, plus every dependency and file hash



No activation: uv run does it

```python
# main.py
from rich.console import Console

console = Console()
console.print("Run TB-014: [bold green]OK[/]")
```

```
uv run python main.py
```

```
Run TB-014: OK
```




What goes into Git, and what does not

Commit

- `pyproject.toml`
- `uv.lock`
- `.python-version`
- your `.py` files and `README.md`

Do not commit

- `.venv/`, big and machine specific
- `__pycache__/`, generated

```
uv sync
```

A colleague clones the repository, runs `uv sync`, and gets your environment.

Exercise 08: A uv project

`exercises/python/08-uv-project`



Exercise 09: Automation capstone

`exercises/python/09-automation-capstone`

Read all bench logs in a folder, summarise each run, write one report file.

Version control



The final_v3 problem

```
\\fileserver\compressors\testbench\
    analyse_run.py
    analyse_run_final.py
    analyse_run_final_v2.py
    analyse_run_final_v3_JK_edit.py
    analyse_run_final_v3_USE_THIS_ONE.py
```

- Which one produced last week's report?
- Who changed the conversion factor, and why?
- Two people edited at the same time: whose version wins?



What we want from a version control tool



History

Every saved state, kept



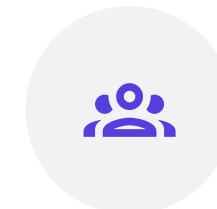
Who and why

Author, date and a message



Undo

Go back to any earlier state



Collaborate

Work in parallel, then combine

Git is the tool, Azure DevOps is the host

Git

- Free program on your laptop
- Keeps the history of a folder
- Works offline
- Same tool everywhere

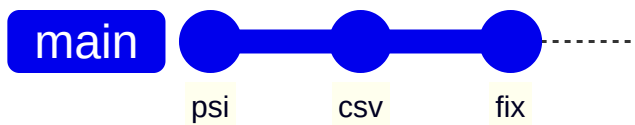
Azure DevOps Repos

- Server that stores a shared copy
- Sign-in, permissions, backup
- Pull requests and review
- Pipelines, later in this training

Comparable to MATLAB and a network drive, except Git tracks changes and the host adds teamwork.

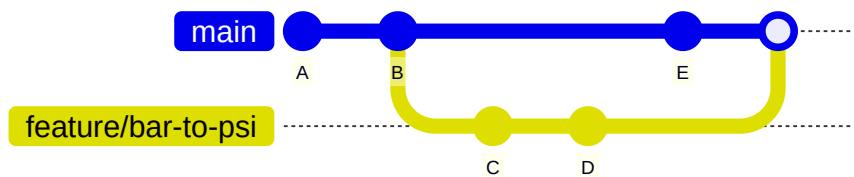
Repository and commit

- **Repository**: a project folder plus its full history
- **Commit**: one saved snapshot of the project
- Each commit has a message, an author, a date and a parent
- Commits form a chain, newest at the end

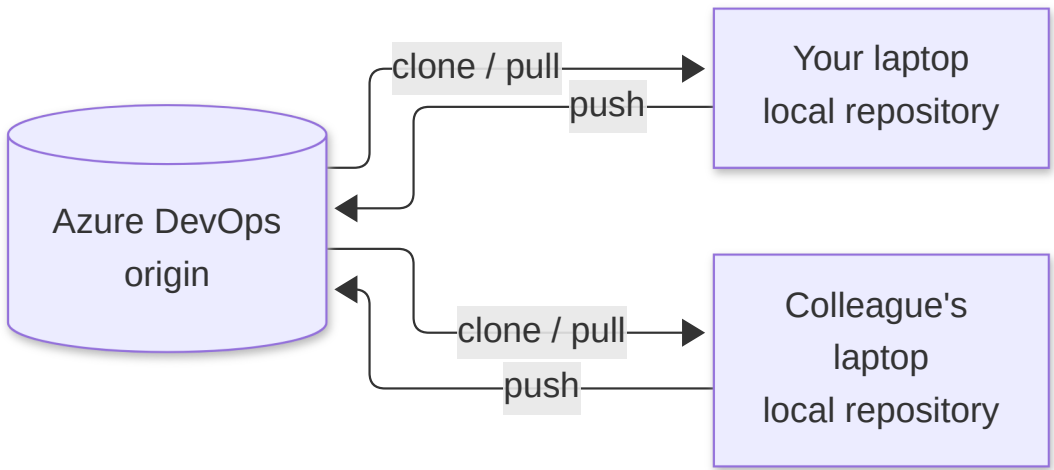


Branch: a movable label

- A branch is a name that points at a commit
- New commits move the label forward
- Work on a branch does not touch `main`
- Merge it back when it is ready

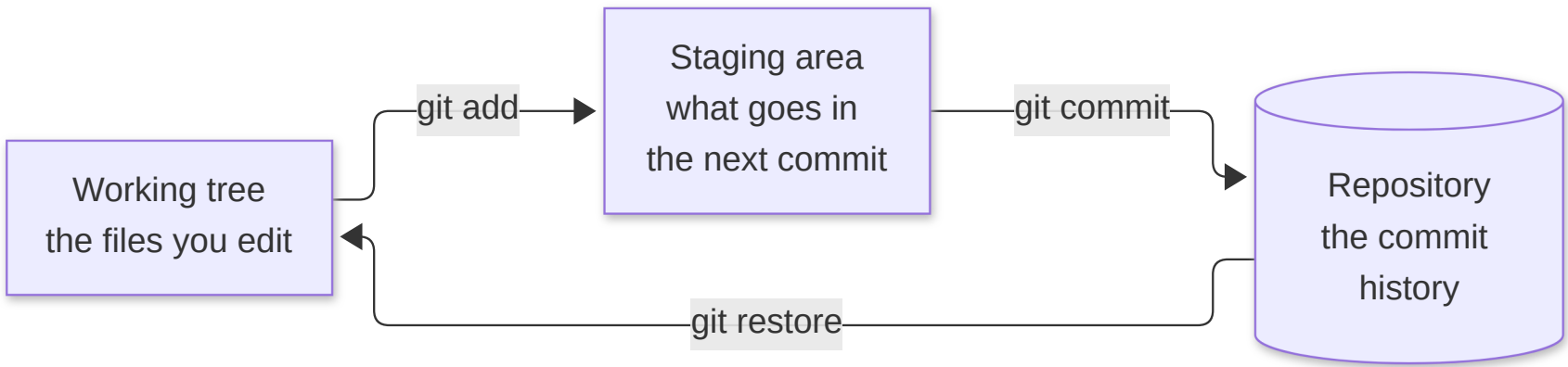


Remote: another copy, somewhere else



- Every laptop holds the **full history**; the remote is the shared meeting point
- By convention the remote is called `origin`

Three areas on your laptop



- You choose what goes into a commit by **staging** it first
- `git status` tells you which area each change is in

Vocabulary recap

Term	In one line
Repository	Project folder with its history
Commit	A saved snapshot with a message
Staging area	Changes selected for the next commit
Branch	A movable label on a commit
Remote	A copy of the repository on a server
Pull request	A request to merge a branch, with review



Git on the command line



Tell Git who you are

```
git config --global user.name "Your Name"
git config --global user.email "you@atlascopco.com"
git config --global init.defaultBranch main
git config --global --list
```

- Do this once per laptop
- Use the same email as your Azure DevOps account
- The name and email end up in every commit you make

Start a repository: init and status

```
mkdir testbench
cd testbench
git init -b main
git status
```

```
Initialized empty Git repository in C:/Users/you/testbench/.git/
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```




Stage and commit

```
git add conversions.py
git status
git commit -m "Add psi to bar conversion"
```

```
Changes to be committed:
    new file:   conversions.py

[main (root-commit) a055887] Add psi to bar conversion
 1 file changed, 2 insertions(+)
 create mode 100644 conversions.py
```

- `git add .` stages everything in the folder; check `git status` first



A good commit message

Avoid

```
update
```

```
fixed stuff
```

```
analyse_run final
```

Prefer

```
Fix psi to bar factor
```

```
Add run filter to analyse_run
```

```
Read measurements as UTF-8
```

Short, imperative, says what the commit does. One logical change per commit.



Read the history with log

```
git log --oneline
```

```
9b221ad Add first measurement
a055887 Add psi to bar conversion
```

```
git log            # full format: author, date, message
git log --graph --oneline --all   # branches too
```




See what changed with diff

```
git diff            # working tree vs staging area
git diff --staged   # staging area vs last commit
```

```
--- a/conversions.py
+++ b/conversions.py
@@ -1,2 +1,6 @@
 def psi_to_bar(psi):
     return psi * 0.0689476
+
+
+def bar_to_psi(bar):
+    return bar / 0.0689476
```




Tip: `q` leaves the pager

When output is long, Git shows it in a pager and the prompt seems stuck.

- `q` quits
- Space scrolls down, `b` scrolls up
- `/word` searches

```
9b221ad Add first measurement
a055887 Add psi to bar conversion
...
(END)
```




Exercise time: first commit

`exercises/git/01-first-commit`

```
uv run python exercises/git/setup_exercise.py 1
```



Undo a change: restore

```
git restore conversions.py            # discard edits since last commit
git restore --staged conversions.py   # unstage, keep the edits
```

- `git restore <file>` throws your edits away: there is no undo for that
- Committed work is safe: you can always get it back



Look at an old version

```
git show a055887                       # one commit: message and diff
git show HEAD~1:conversions.py         # a file as it was one commit ago
git restore --source=HEAD~1 conversions.py
```

- `HEAD` is the latest commit, `HEAD~1` the one before it
- You can use the short ID from `git log --oneline` instead
- Restoring an old version shows up as a normal change: commit it or discard it



Exercise time: history

`exercises/git/02-history`

```
uv run python exercises/git/setup_exercise.py 2
```



Branches: switch and create

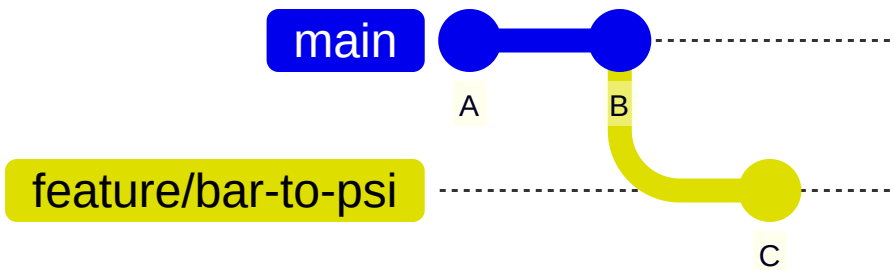
```
git switch -c feature/bar-to-psi   # create and move to a new branch
git branch                         # list local branches
git switch main                    # go back
git branch -d feature/bar-to-psi   # delete after merging
```

```
  feature/bar-to-psi
* main
```

- The `*` marks the branch you are on



Merge: fast-forward




```
git switch main
git merge feature/bar-to-psi
```

```
Updating 9b221ad..1cd9279
Fast-forward
```

Merge: merge commit

```
git merge feature/rounding
git log --oneline --graph
```

```
*   bc8a0a7 Merge branch 'feature/rounding'
|\
| * 3a2e549 Add analyse_run
* | 247671c Add readme.py
|/
* 1cd9279 Add bar to psi conversion
```

- Both branches moved, so Git creates a commit that joins them
- It may open an editor for the message: save and close it



When two branches edit the same lines

```
git merge feature/a
```

```
CONFLICT (content): Merge conflict in conversions.py
Automatic merge failed; fix conflicts and then commit the result.
```

```
def psi_to_bar(psi):
<<<<<<< HEAD
    return psi * 0.068948
=======
    return psi * 0.06895
>>>>>>> feature/a
```




Resolving a conflict

1

OPEN THE FILE

VS Code highlights the conflict and offers Accept Current, Incoming or Both.

2

DECIDE

Keep one side or combine them, and delete the three marker lines.

3

STAGE AND COMMIT

`git add conversions.py`, then `git commit`.

Changed your mind? `git merge --abort` returns to before the merge.

Exercise time: branches

`exercises/git/03-branches`

```
uv run python exercises/git/setup_exercise.py 3
```



Remotes and Azure DevOps



Clone a repository

```
git clone https://dev.azure.com/<organisation>/<project>/_git/<training-repo>
cd <training-repo>
git status
```

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

- The first time, a browser window asks you to sign in (Git Credential Manager)
- The URL is on the **Clone** button of the repository in Azure DevOps



The remote is called origin

```
git remote -v
```

```
origin  https://dev.azure.com/<organisation>/<project>/_git/<training-repo> (fetch)
origin  https://dev.azure.com/<organisation>/<project>/_git/<training-repo> (push)
```

- `clone` created the remote named `origin` for you
- `origin/main` is your laptop's memory of where `main` was on the server
- A local repository made with `git init` has no remote until you add one



Share a branch: push -u

```
git switch -c feature/bar-to-psi
git add conversions.py
git commit -m "Add bar to psi conversion"
git push -u origin feature/bar-to-psi
```

```
To https://dev.azure.com/<organisation>/<project>/_git/<training-repo>
 * [new branch]      feature/bar-to-psi -> feature/bar-to-psi
branch 'feature/bar-to-psi' set up to track 'origin/feature/bar-to-psi'.
```

- `-u` is needed only the first time; later a plain `git push` is enough



Fetch or pull

git fetch

- Downloads new commits from the server
- Does not change your files or branches
- Safe at any time
- Then look: `git log origin/main`

git pull

- Fetch, then merge into your current branch
- Updates your files
- Do it on `main` before you start new work
- Commit or restore your edits first

Exercise time: Azure DevOps remote

`exercises/git/04-azure-devops-remote`



The pull request flow



- A pull request (PR) proposes merging a branch into `main`
- Colleagues read the diff and comment before anything changes on `main`
- It is a feature of Azure DevOps, not of Git

In Azure DevOps: Repos, Pull requests

1

CREATE

New pull request: source `feature/...`, target `main`, add a title and description.

2

REVIEW

Reviewers read the Files tab and comment on lines.

3

APPROVE

A reviewer approves, or waits for changes.

4

COMPLETE

Choose the merge type, tick **Delete source branch**, confirm.



Merge types when you complete

Merge type	Result on `main`
Merge (no fast-forward)	All commits kept, plus a merge commit
Squash merge	One single commit for the whole PR
Rebase and fast-forward	Commits replayed, straight line
Semi-linear merge	Rebase, then a merge commit

- For small feature branches, squash merge keeps `main` readable
- Use what the team agrees on; it can be set per repository



After the merge: update local

```
git switch main
git pull
git branch -d feature/bar-to-psi
```

```
Updating 9b221ad..5c0e1fa
Fast-forward
 conversions.py | 3 +++
 1 file changed, 3 insertions(+)
Deleted branch feature/bar-to-psi (was 1cd9279).
```

- Your laptop does not know about the merge until you pull



Exercise time: pull request

`exercises/git/05-pull-request`



The daily cycle




```
git switch main; git pull
git switch -c feature/x
git add <file>; git commit -m "message"
git push -u origin feature/x
# create, review and complete the PR in Azure DevOps, then repeat
```

Keep files out: .gitignore

```
# .gitignore
.venv/
__pycache__/
.ipynb_checkpoints/
data/raw/
```

```
git add .gitignore
git commit -m "Ignore virtual environment and raw data"
```

- One pattern per line; commit the file itself so everyone shares it
- Never commit passwords, tokens or large raw data
- Day 2 uses it for data files and notebooks



Three ways to use Git

	Command line	VS Code	SourceTree
Advantages	Every feature, same everywhere, easy to copy into notes and scripts	Already open, diff and conflict views built in	Clear graph of branches, point and click
Disadvantages	Commands to learn, easy to mistype	Hides the commands, fewer rare features	Extra install, less common on managed laptops
Use it when	You need the full power, or help from a colleague or a doc	Daily edit, commit, push work next to the code	You want to browse a complex history visually

The same cycle in VS Code

Command line	Source Control view
`git status`	List of changed files
`git diff`	Click a file to see the diff
`git add`	Plus icon next to a file
`git commit -m "..."`	Type a message, press Commit
`git switch -c`	Branch name at the bottom left
`git push` / `git pull`	Sync Changes and the three-dot menu



Exercise time: Git in VS Code

`exercises/git/06-git-in-vscode`



Branching models

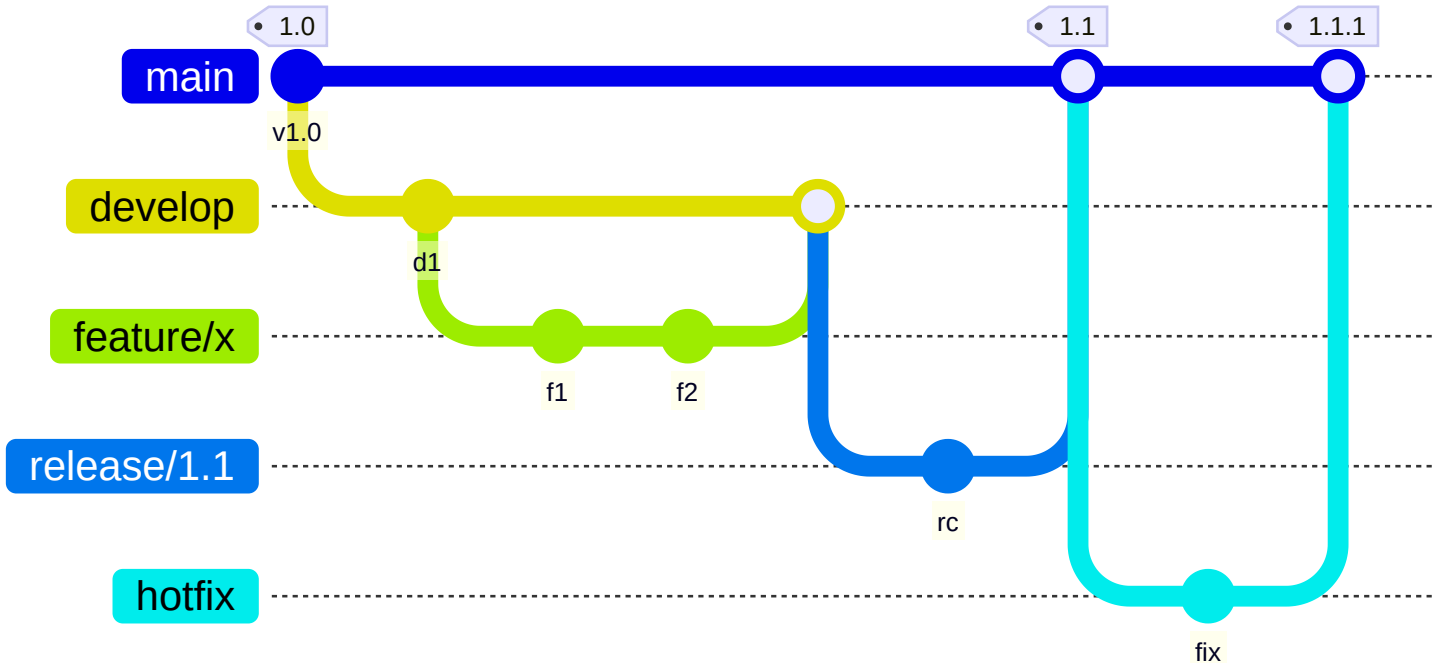


Why a model?

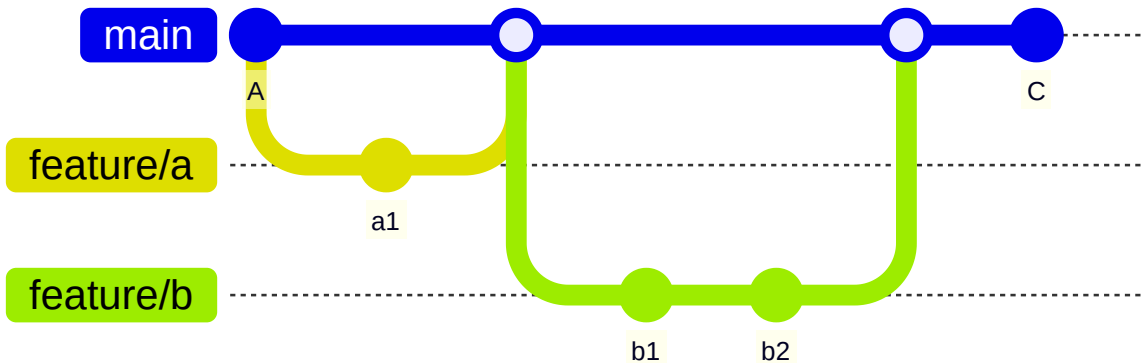
- Git does not tell a team **how** to use branches
- Without agreement: long-lived branches, big merges, many conflicts
- A branching model answers: where does new work start, and where does it end up?
- Two well-known answers: GitFlow and trunk-based development



GitFlow


1.0
1.1
1.1.1
main
v1.0
develop
d1
feature/x
f1
f2
release/1.1
rc
hotfix
fix

Trunk-based development



- One long-lived branch: `main`
- Short branches for hours or a few days, merged through a pull request

GitFlow versus trunk-based

	GitFlow	Trunk-based
Branches	main, develop, feature, release, hotfix	main plus short feature branches
Merges	Large, less often, more conflicts	Small and frequent
Fits	Versioned products, several releases in support	Continuous delivery, scripts, analyses, small teams
Cost	Many branches to keep in sync	Needs discipline: small changes, review, tests



Our recommendation

1

TRUNK-BASED

One long-lived branch, `main`, always working.

2

SHORT-LIVED BRANCHES

One branch per small change, named `feature/...` or `fix/...`.

3

PULL REQUESTS

Merge into `main` only through a reviewed PR.

On day 3 we protect `main` with a branch policy so this is enforced.



Habits that make it work

- Keep changes **small**: one idea per branch, one reviewer-friendly PR
- Pull `main` often, and before you start a new branch
- Delete the branch after the PR is completed
- Do not commit to `main` directly, even when you could
- Talk to your colleague before you both edit the same file



Exists, not today

git rebase

Replay your commits on top of the latest `main` for a straight history.

git cherry-pick

Copy a single commit from one branch to another.

git stash

Park uncommitted edits and bring them back later.

Resolving large conflicts and release strategies come later, when you need them.

Wrap-up

What you can do now: Python

Python

- Run a script from PowerShell
- Keep a virtual environment per project with uv
- Use functions, conditionals and loops
- Import a package and run it from VS Code

Working with code

- Read a stack trace and find the failing line
- Automate a task that reads and writes files



What you can do now: Git



Clone an Azure DevOps repository, commit on a branch, push, merge through a PR and pull the result. Plus: read the history, undo a change, and recognise a merge conflict.

Git cheat sheet

Local

```
git status
git add <file>
git commit -m "message"
git log --oneline
git diff
git restore <file>
git switch -c <branch>
git merge <branch>
```

Remote

```
git clone <url>
git remote -v
git pull
git push -u origin <branch>
git fetch
git branch -d <branch>
git merge --abort
```




Before day 2

1

WORKING UV ENVIRONMENT

Create a project, add a package, run a script from PowerShell and from VS Code.

2

LANGUAGE FUNDAMENTALS

Redo the exercises you found hard: functions, loops, files, exceptions.

3

CLONE, COMMIT, PUSH, PULL

Do one full cycle on Azure DevOps, including a pull request.



Further reading

- **Pro Git**, free book: `git-scm.com/book`
- **Learn Git Branching**, interactive visual game: `learngitbranching.js.org`
- **The Python Tutorial**, official: `docs.python.org/3/tutorial`
- **uv documentation**: `docs.astral.sh/uv`
- Azure DevOps Repos documentation: `learn.microsoft.com/azure/devops/repos`



Day 2 and day 3

Day 2: data analysis

- pandas and NumPy
- SciPy and Matplotlib
- scikit-learn
- `.gitignore` for data files, nbstripout for notebooks

Day 3: quality and automation

- pytest, ruff, type checking
- pre-commit
- Azure DevOps pipelines and branch policies
- GitHub Copilot and coding agents





Thank you

See you on day 2

